

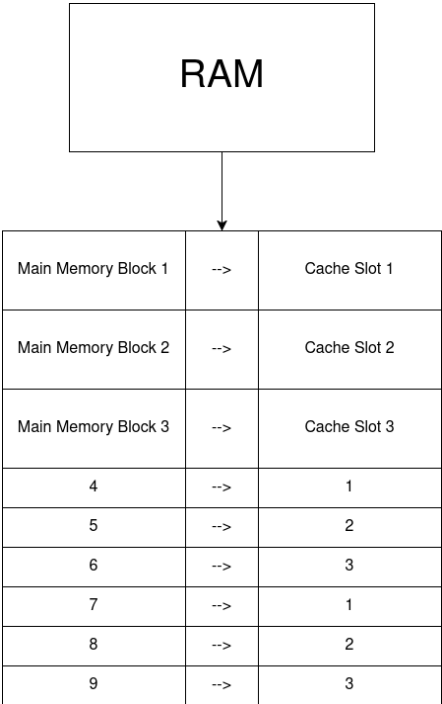
With the aid of a diagram, show how a cache-tag RAM is used to implement a direct-mapped cache memory. Discuss the advantages and disadvantages of a direct-mapped cache over an associatively mapped cache.

Direct-mapped cache

In the directly-mapped model of cache, each RAM address is mapped to exactly one cache memory address. For example, let’s say a CPU needs to fetch the data at the memory address 00010110. Splitting it up, the address consists of a tag, index, and offset.

0001	01	10
Tag	Index	Offset

The tag is crucial because indices are not unique in terms of content (data). An index might- and will most likely- be shared with other RAM addresses because of the cyclical nature of RAM to cache memory assignment. As previously stated, direct-mapped cache assigns exactly one slot to each secondary storage address. The diagram to the right demonstrates said assignment. The tag solves this problem by assigning an additional piece



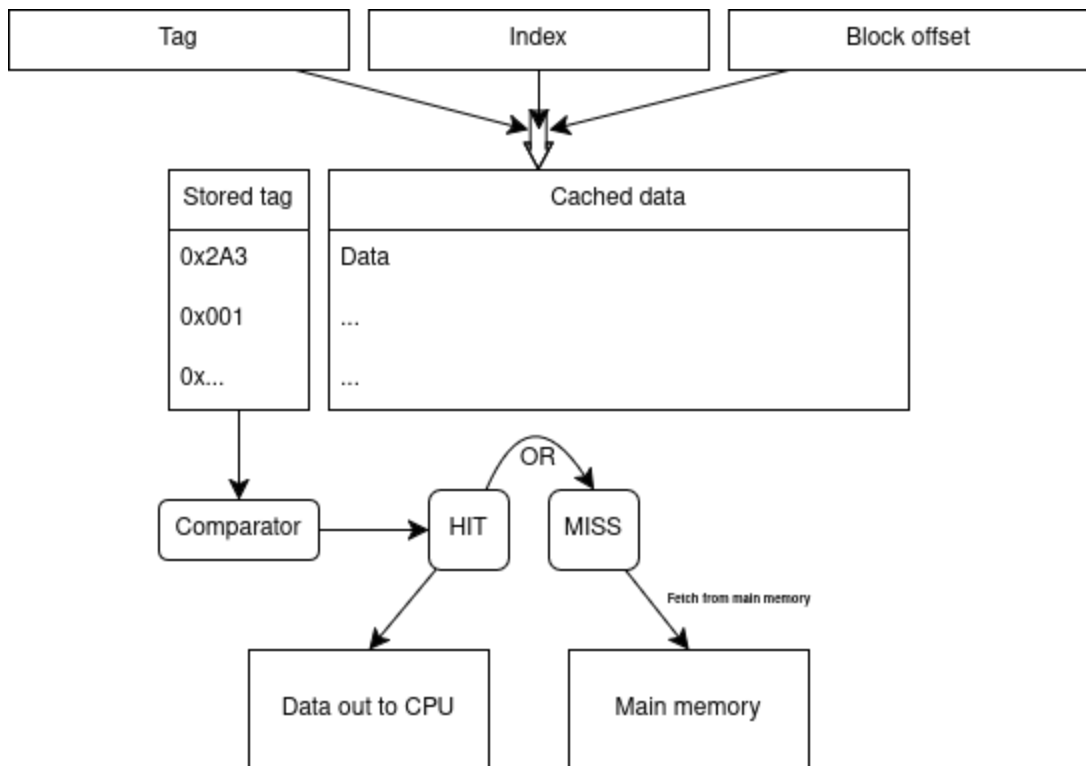
of data that allows the data to be uniquely identified. Both the tag and index are searched in sequence in order to find the target address. This means that if either the index or the tag are inconsistent with the target, the search is marked as a miss (it isn't currently stored in cache) and the CPU would have to fetch it from main memory.

The index bits are derived directly from the RAM address. That is to say that if a RAM address has the index of 11 (001011 in binary), the cache slot would place that data into slot 11 in cached memory. This direct lookup allows for quick indexing of memory addresses.

The offset is needed for memory addresses because entire blocks of bytes are held within each address (in cache, not RAM), not just one specific address. The offset then is required in order to specify which byte the CPU is looking for. For example, an offset of 3 gets the third byte of a block from memory.

Associatively mapped cache

In associatively mapped cache, any blocks of data from main memory can be placed in any cache line. It does not have pre-assigned slots like directly mapped cache does. The best way to understand how memory is fetched in an associative cache architecture is to think of the CPU as a networking node broadcasting to every other node within that network. When the CPU is looking for data, it can't look at each index in the cache to see if the data is there since there are no indexes in associatively mapped cache (again, no fixed positions). The cache controller has to compare the target tag with each tag currently in the cache in parallel.



The difference between the two comes down to how complex the hardware implementations are. For directly mapped cache, the hardware is very simple, with a single comparator and no real decisions. This makes it both cheap to implement as well as much faster than associatively mapped cache. The speed is a great advantage when considering directly mapped architecture for L1 cache. Associatively mapped cache is much more expensive because of its more complex nature. In order to implement it you'd need as many comparators as cache lines in cache, all operating simultaneously. This makes a fully associatively mapped cache impractical for large cache sizes. As with many choices made in computer architecture and design, the correct choice comes down to use case.